Trabalho Prático: Divisão e Conquista

Projeto e Análise de Algoritmos

Amanda Uchoa [nome do(a) colega]

Universidade de Fortaleza (UNIFOR), 2026

1 Introdução

Este relatório apresenta a implementação e a análise de dois problemas resolvidos com divisão e conquista (D&C): o *subarranjo de soma máxima* (Etapa 1) e o *par de pontos mais próximos* (Etapa 2). Cada um foi comparado com uma solução de força bruta e, na Etapa 1, também com a versão $O(n)$ (Kadane). Todo o código está em Python 3, no repositório junto com as instruções de execução (`README.md`), os casos de teste (`casos_de_teste.json`) e as medições brutas (`resultados/medicoes.csv`).

Os algoritmos de ordenação vistos em aula (merge sort e quick sort) foram reaproveitados na Etapa 2. O *quick sort* faz a ordenação inicial por x, e a função `combinacao` do *merge sort* mantém os pontos ordenados por y durante a recursão. Duas mudanças foram necessárias. A versão da aula da `combinacao` não incrementava `i` e entrava em laço infinito, e isso foi corrigido. O quick sort passou a sortear o pivô, porque com o pivô fixo no fim um vetor já ordenado cai no pior caso $O(n^2)$ e estoura o limite de recursão do Python.

2 Etapa 1: Subarranjo de Soma Máxima

Entrada e saída. A entrada é n seguido de n inteiros. A saída é a soma máxima e os índices (a partir de 0) de início e fim. O intervalo tem pelo menos um elemento, então um vetor só de negativos devolve o maior negativo. **Regra de desempate:** maior soma; se empatar, menor início; se empatar de novo, menor fim. As três implementações seguem a mesma regra, e os testes aleatórios conferem que elas devolvem exatamente a mesma tripla.

Força bruta. Para cada início i, o laço interno avança o fim j e reaproveita a soma anterior (`soma += v[j]`). O número de intervalos testados é $\sum_{i=0}^{n-1}(n-i) = \frac{n(n+1)}{2} = \Theta(n^2)$. A melhor resposta só é trocada quando a soma é *estritamente* maior, e isso já garante o menor i e o menor j.

Divisão e conquista. O vetor é dividido ao meio, como no merge sort, e cada metade é resolvida recursivamente. Depois, em $O(n)$, calcula-se o melhor subarranjo que *cruza o meio*: anda-se do meio para a esquerda guardando a melhor soma (com `>=`, para ficar com o início mais à esquerda nos empates) e do meio+1 para a direita (com `>`, para ficar com o fim mais à esquerda). A resposta é o melhor dos três candidatos. Todo subarranjo está inteiro na esquerda, inteiro na direita ou cruza o meio, e cada candidato já é o melhor de sua categoria pela regra de desempate. Por isso a combinação está correta. A recorrência é

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$

Pelo Teorema Mestre ($a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$, caso 2), $T(n) = \Theta(n \log n)$. Na árvore de recursão, cada um dos $\log_2 n$ níveis custa cn no total, o que dá o mesmo resultado.

Questão desafio: Kadane, $O(n)$. Seja M_j a melhor soma de um intervalo que *termina* em j. Vale $M_j = \max(v_j,\ M_{j-1} + v_j)$: o intervalo continua se $M_{j-1} \geq 0$ e recomeça em j se $M_{j-1} < 0$. Usar $\geq$ em vez de $>$ faz com que, quando $M_{j-1} = 0$, o intervalo continue. A soma é a mesma, mas o início é menor, como pede a regra. A resposta é o melhor M_j, com um único laço: $T(n) = \Theta(n)$.

3 Etapa 2: Par de Pontos Mais Próximos

Força bruta. Compara os $\binom{n}{2} = \Theta(n^2)$ pares.

Divisão e conquista.

1. Ordenação inicial por x (desempate por y) com o quick sort de pivô aleatório: $O(n \log n)$ esperado.
2. Divisão na mediana: os $\lfloor n/2 \rfloor$ primeiros vão para a esquerda e o resto para a direita. A linha vertical fica em $x_m = x$ do ponto do meio. Dividir pela *posição* na lista ordenada, e não pelo valor de x, garante metades de tamanho equilibrado mesmo quando muitos pontos têm o mesmo x (caso de teste E2-03).
3. Cada lado é resolvido recursivamente e $\delta = \min(\delta_E, \delta_D)$. Cada chamada também devolve seus pontos *ordenados por* y, e as duas listas são juntadas pela `combinacao` do merge sort em $O(n)$.
4. Monta-se a faixa $\{p : |p_x - x_m| < \delta\}$, já ordenada por y. Cada ponto é comparado com **no máximo os 7 seguintes** da faixa, parando antes se a diferença em y já for $\geq \delta$.
5. **Limiar:** se o subproblema tem até L pontos, ele é resolvido por força bruta. Escolhemos $L = 8$ (Seção 5.3).

O passo 3 evita ordenar por y em cada nível, o que daria $\Theta(n \log^2 n)$. Com ele, dividir (fatiar a lista), juntar por y e varrer a faixa custam $O(n)$ por chamada:

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(n) = \Theta(L^2) \text{ se } n \leq L.$$

A árvore tem $\log_2(n/L)$ níveis de custo cn e n/L folhas de custo $\Theta(L^2)$. Logo $T(n) = \Theta(n \log \frac{n}{L} + nL) = \Theta(n \log n)$ para L constante, e somando a ordenação inicial o total continua $\Theta(n \log n)$.

Por que bastam 7 comparações por ponto na faixa. Seja p um ponto da faixa e considere os pontos que vêm *depois* dele na ordem por y e que podem estar a menos de δ de p. Todos estão no retângulo $R = [x_m - \delta,\ x_m + \delta] \times [p_y,\ p_y + \delta]$, de tamanho $2\delta \times \delta$. Divida R em dois quadrados $\delta \times \delta$, um de cada lado da linha. Os pontos da metade esquerda estão todos a uma distância $\geq \delta_E \geq \delta$ uns dos outros. Em um quadrado $\delta \times \delta$ cabem no máximo 4 pontos com essa propriedade: dividindo o quadrado em 4 quadradinhos $\frac{\delta}{2} \times \frac{\delta}{2}$, dois pontos no mesmo quadradinho estariam a no máximo $\frac{\delta}{\sqrt{2}} < \delta$, o que é uma contradição. O mesmo vale para a direita. Assim, R tem no máximo $4 + 4 = 8$ pontos, incluindo o próprio p, e por isso só é preciso olhar **os 7 seguintes**. Pontos em cima da linha pertencem a um lado só (o da sua posição na lista), então a contagem continua valendo com coordenadas x repetidas. Como a varredura é feita para todo p, cada par próximo é encontrado a partir do ponto de menor posição na ordem por y.

4 Testes

A bateria (`testes.py`, 15 testes do `unittest`, todos passando) tem três partes:

1. **Casos fixos** (`casos_de_teste.json`): 18 da Etapa 1 e 12 da Etapa 2, cada um com entrada, saída esperada e **justificativa** escrita no próprio arquivo. Os casos da Etapa 2 rodam a D&C com os limiares 1, 2, 3, 4, 8, 12 e 16. Com limiares pequenos, mesmo entradas pequenas passam pela recursão e pela faixa.
2. **Testes aleatórios contra a força bruta**: 3000 vetores na Etapa 1 (valores em faixas pequenas como $[-2, 2]$, que geram muitos empates) e 1800 conjuntos de pontos na Etapa 2 (inteiros em grades pequenas, que geram pontos repetidos e x iguais, e também reais). Também

verificamos se a soma devolvida é de fato a soma do intervalo devolvido e se o par devolvido realmente tem a distância informada.

3. **Linha de comando**: os scripts são executados com a entrada pelo `stdin`, como o professor fará.

Casos	Motivo
E1-01 a 03	**Tamanho 1** (positivo, negativo e zero): é o caso base da recursão.
E1-04, 05, 18	**Só negativos**: a resposta é o maior negativo. Uma implementação que começa com `melhor = 0` erra aqui.
E1-08 a 11	**Empates**: intervalos disjuntos, intervalos encaixados, zeros e vetor todo de zeros. O E1-10 ($[0, 0, 5, 0] \to (5, 0, 2)$) testa se o Kadane continua o intervalo quando a soma atual é 0.
E1-12 a 15	Resposta cruzando o meio, só na metade direita, só na esquerda, e atravessando um negativo.
E1-16	Soma maior que 2^{31}: em Python não há estouro, mas em C/Java haveria.
E2-01	$n = 2$, a menor entrada válida.
E2-02	**Pontos repetidos**: $\delta = 0$ e a faixa fica vazia.
E2-03, 04	Todos com o mesmo x (a linha passa em cima de todos) ou o mesmo y.
E2-05, 11	Par mais próximo com um ponto de cada lado da linha: só é achado na faixa.
E2-06, 07, 12	Coordenadas negativas, reais e grandes.
E2-08, 09	Empates de distância e grade 3×3 (muitos pontos dentro da faixa).

Tabela 1: Casos de borda e por que foram incluídos (o texto completo está no JSON).

5 Etapa 3: Análise Experimental

Metodologia. Para cada $n \in \{100, 200, 400, 800, 1600, 3200, 6400\}$ foram feitas **5 execuções**, cada uma com uma entrada aleatória nova: inteiros em $[-100, 100]$ na Etapa 1 e pontos reais em $[0, 10^4]^2$ na Etapa 2. O tempo foi medido com `time.perf_counter` e o coletor de lixo desligado durante a medição. As tabelas mostram média ± desvio padrão. Para entradas pequenas ($n \leq 256$), cada execução repete a chamada várias vezes e divide o tempo, porque uma chamada sozinha é rápida demais para o relógio. **Máquina:** Intel Xeon @ 2,10 GHz (2 núcleos, máquina virtual em nuvem), 7,8 GB de RAM, Linux 6.18, Python 3.13.18. Por ser uma VM compartilhada, há algum ruído, visível nos desvios padrão.

n	Força bruta (ms)	Divisão e conquista (ms)	Kadane O(n) (ms)
100	0.283 ± 0.043	0.131 ± 0.031	0.025 ± 0.009
200	0.858 ± 0.019	0.222 ± 0.019	0.032 ± 0.000
400	4.369 ± 1.623	0.497 ± 0.031	0.065 ± 0.010
800	16.740 ± 3.135	1.012 ± 0.037	0.122 ± 0.018
1600	71.553 ± 17.867	2.456 ± 0.539	0.265 ± 0.067
3200	272.306 ± 18.791	6.858 ± 3.437	0.463 ± 0.046
6400	1111.754 ± 48.066	8.698 ± 0.149	0.905 ± 0.069

Tabela 2: Etapa 1: tempo médio em milissegundos (5 execuções).

n	Força bruta (ms)	Divisão e conquista (ms)
100	0.997 ± 0.254	0.470 ± 0.092
200	3.145 ± 0.692	0.749 ± 0.023
400	11.927 ± 1.082	1.679 ± 0.058
800	55.298 ± 14.105	4.244 ± 1.531
1600	199.378 ± 11.407	9.074 ± 2.417
3200	778.067 ± 27.041	18.997 ± 5.057
6400	3164.203 ± 87.405	43.645 ± 10.257

Tabela 3: Etapa 2: tempo médio em milissegundos (5 execuções, D&C com limiar 8).

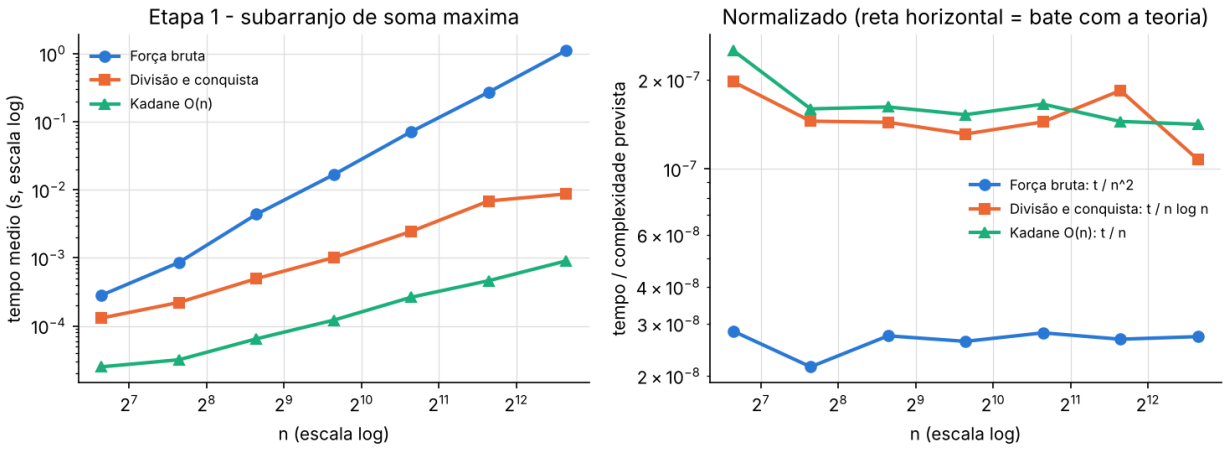


Figura 1: Etapa 1. À esquerda, os tempos (log-log). À direita, o tempo dividido pela complexidade prevista: uma curva horizontal significa que o crescimento bate com a análise.

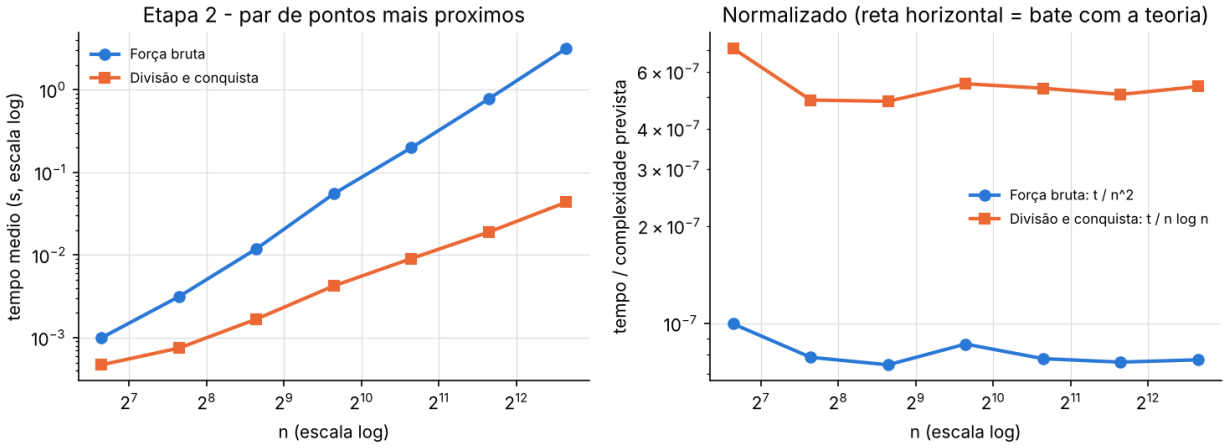


Figura 2: Etapa 2, com as mesmas convenções da Figura 1.

5.1 Comparação com a análise teórica

Em um gráfico log-log, $t = c\,n^k$ vira uma reta de inclinação k. Também é útil olhar quanto o tempo cresce quando n dobra: para n^2 o fator esperado é 4, e para $n \log n$, com n grande, é um pouco mais de 2 (de 3200 para 6400 o fator é $2 \cdot \frac{12{,}64}{11{,}64} \approx 2{,}17$).

- **Força bruta (as duas etapas).** Os gráficos mostram retas de inclinação 2, e as curvas normalizadas t/n^2 são praticamente horizontais. Na Etapa 1, de 3200 para 6400 o tempo foi multiplicado por $1111{,}8/272{,}3 \approx 4{,}1$. Na Etapa 2, de 1600 para 6400 (fator teórico 16) o tempo foi multiplicado por $3164/199{,}4 \approx 15{,}9$. Isso confirma o $\Theta(n^2)$.
- **D&C da Etapa 2.** A curva $t/(n \log n)$ fica quase horizontal a partir de $n \approx 400$. De 1600 para 6400 o tempo cresceu $43{,}6/9{,}07 \approx 4{,}8$ vezes, contra os $4 \cdot \frac{12{,}64}{10{,}64} \approx 4{,}75$ previstos para $n \log n$. A concordância é muito boa.
- **D&C da Etapa 1.** A curva $t/(n \log n)$ cai devagar (de $\approx 2{,}0 \cdot 10^{-7}$ para $\approx 1{,}1 \cdot 10^{-7}$). Isso acontece porque o custo real é $c_1 n \log n + c_2 n$: a recursão faz cerca de $2n$ chamadas de função, e em Python cada chamada (com a criação das tuplas de resposta) custa caro. O termo $c_2 n$ pesa relativamente mais nos n pequenos. (O ponto de $n = 3200$ fica fora da tendência por ruído da máquina virtual: o desvio padrão dele foi de 50% da média.) A complexidade assintótica continua $\Theta(n \log n)$, e o termo de ordem menor só aparece porque a faixa de n medida é pequena.

- **Kadane.** A curva t/n é horizontal a partir de $n = 200$, confirmando $\Theta(n)$. O primeiro ponto ($n = 100$, $\approx 25\,\mu$s) está acima porque, nessa escala, o custo fixo da chamada e da medição domina.

5.2 Quando a força bruta vence e quando a técnica vence

n	Força bruta (μs)	Divisão e conquista (μs)	Kadane O(n) (μs)
2	0.451 ± 0.117	0.715 ± 0.051	0.295 ± 0.032
4	0.816 ± 0.102	2.052 ± 0.242	0.498 ± 0.042
8	2.063 ± 0.113	4.559 ± 0.119	0.959 ± 0.044
16	6.953 ± 0.934	10.220 ± 0.220	2.158 ± 0.543
32	23.123 ± 0.457	22.662 ± 0.121	3.894 ± 0.199
64	95.664 ± 16.814	53.946 ± 5.255	9.184 ± 1.501
128	393.822 ± 28.430	127.551 ± 22.120	18.400 ± 3.978
256	1505.133 ± 115.009	257.914 ± 28.526	35.983 ± 8.770

n	Força bruta (μs)	Divisão e conquista (μs)
2	0.445 ± 0.026	2.273 ± 0.036
4	1.328 ± 0.101	6.394 ± 0.288
8	5.084 ± 0.403	16.485 ± 0.793
16	18.035 ± 0.284	41.918 ± 3.997
32	71.052 ± 1.412	94.132 ± 4.938
64	280.757 ± 4.400	205.899 ± 3.373
128	1191.766 ± 130.965	498.194 ± 76.730
256	5082.719 ± 601.443	1182.133 ± 222.154

Tabela 4: Entradas pequenas, em microssegundos. Em cima, a Etapa 1; embaixo, a Etapa 2 (D&C "pura", com limiar 3).

A complexidade assintótica só diz qual algoritmo vence *para n suficientemente grande*. Para n pequeno, o que decide são as constantes. A força bruta tem um laço muito simples (uma soma ou uma distância por iteração). A D&C paga chamadas recursivas, fatias de lista, a ordenação inicial, a junção por y e a montagem da faixa. Pela Tabela 4:

- **Etapa 1:** a força bruta é mais rápida até $n = 16$, os dois empatam em $n = 32$ (23,1 contra 22,7 μs), e a partir de $n = 64$ a D&C vence. Em $n = 6400$ ela é cerca de 128 vezes mais rápida. O Kadane vence os dois em *todos* os tamanhos, porque faz menos trabalho e também tem constante baixa.
- **Etapa 2:** o ponto de cruzamento está entre $n = 32$ e $n = 64$. Em $n = 6400$ a D&C é cerca de 72 vezes mais rápida que a força bruta.

Igualando $c_b n^2/2 \approx c_d\, n \log_2 n$, o cruzamento ocorre em $n/\log_2 n \approx 2c_d/c_b$. Os valores medidos indicam que, em Python, uma "unidade de trabalho" da D&C custa algumas vezes mais que uma iteração da força bruta.

5.3 Escolha do limiar na Etapa 2

O resultado anterior sugere uma melhoria: abaixo de certo tamanho, é melhor parar a recursão e usar a força bruta. Testamos 12 limiares L (de 2 a 128) com $n = 1600$ e $n = 6400$, usando as **mesmas 15 entradas** para todos os limiares. Aqui foram mais que 5 execuções porque as diferenças entre limiares são pequenas.

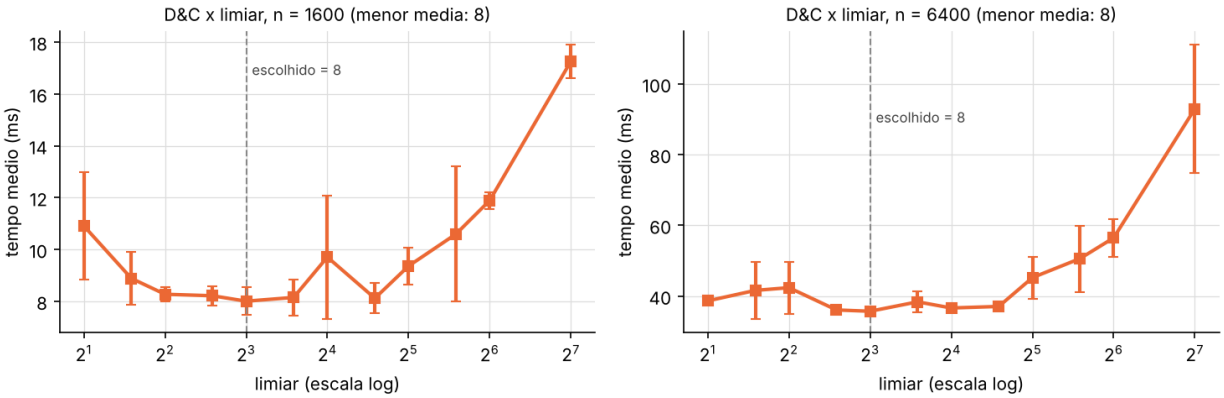


Figura 3: Tempo médio da D&C em função do limiar (barras = desvio padrão). A linha tracejada marca o valor escolhido.

- **Limiar grande demais ($L \geq 32$):** o tempo sobe rápido. Com $L = 128$ fica cerca de 2,2 vezes maior (17,3 contra 8,0 ms em $n = 1600$) porque as folhas, de custo $\Theta(L^2)$, passam a dominar. É o termo nL da análise.
- **Limiar pequeno demais ($L = 2, 3$):** a recursão vai até subproblemas de 2 ou 3 pontos, e o custo das chamadas é maior que as poucas comparações economizadas. Com $L = 2$ e $n = 1600$ o tempo foi 36% maior que com $L = 8$.
- **Entre 6 e 24** há um "platô" em que as diferenças ficam dentro do desvio padrão. $L = 8$ teve a **menor média nos dois tamanhos** (8,00 ms e 35,7 ms) e baixa variação, por isso foi o escolhido. Ele também é coerente com a Tabela 4: com até ≈16 pontos, a força bruta é claramente mais rápida que a D&C pura.

Um detalhe ajuda a interpretar o gráfico: como a lista é sempre dividida ao meio, só importam os tamanhos que realmente aparecem na recursão. Com $n = 6400$ os tamanhos são $6400, 3200, \ldots, 50, 25, 12/13, 6/7$. Assim, $L = 16$ e $L = 24$ geram *exatamente* a mesma árvore (folhas de 12 e 13 pontos), e a diferença medida entre eles (36,6 e 37,1 ms) dá uma ideia do tamanho do ruído da medição.

6 Conclusão

As três implementações da Etapa 1 e as duas da Etapa 2 passaram em todos os testes, inclusive nos casos de borda e nas milhares de comparações aleatórias com a força bruta. As medições confirmaram as recorrências: $\Theta(n^2)$ para a força bruta, $\Theta(n \log n)$ para a D&C ($T(n) = 2T(n/2) + \Theta(n)$) e $\Theta(n)$ para o Kadane. Também ficou claro que a complexidade assintótica não conta a história toda. Para entradas pequenas ($n \lesssim 32$) a força bruta é mais rápida, e é exatamente isso que justifica usar um limiar ($L = 8$) na Etapa 2, aproveitando o melhor das duas abordagens.